

Exploratory Data Analysis

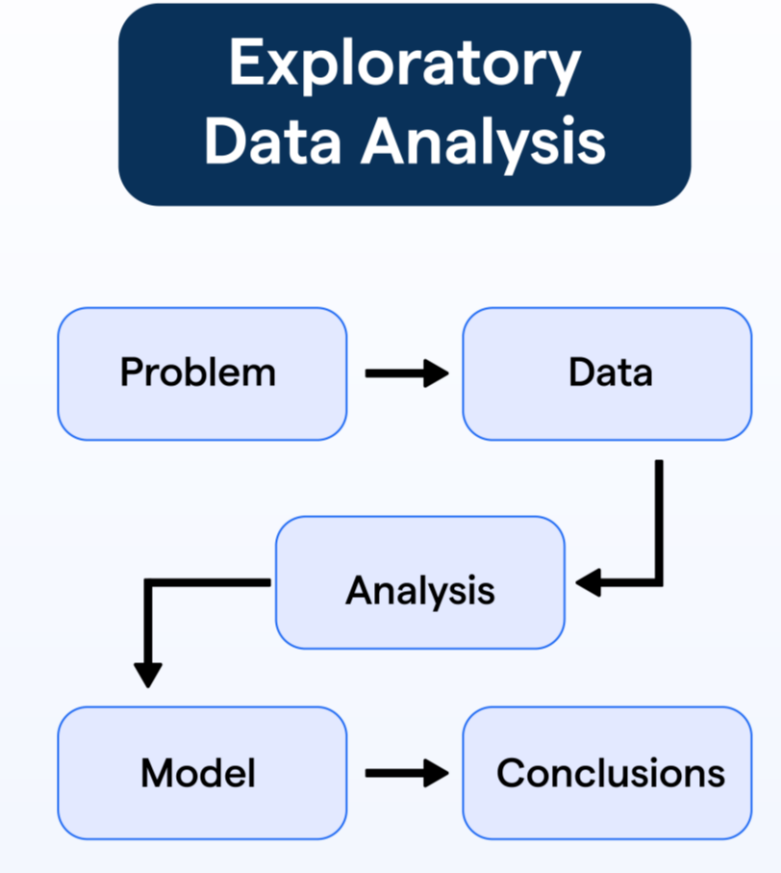
Pandas

Lecture 6 – HCCDA-AI

Imran Nawar

Exploratory Data Analysis

- Exploratory Data Analysis (EDA) is the process of examining a dataset, often with visual methods, to summarize its main characteristics.
- It is a crucial step in the data analysis workflow, used to gain a deep understanding of the data before modelling.



Using EDA, data scientists can:

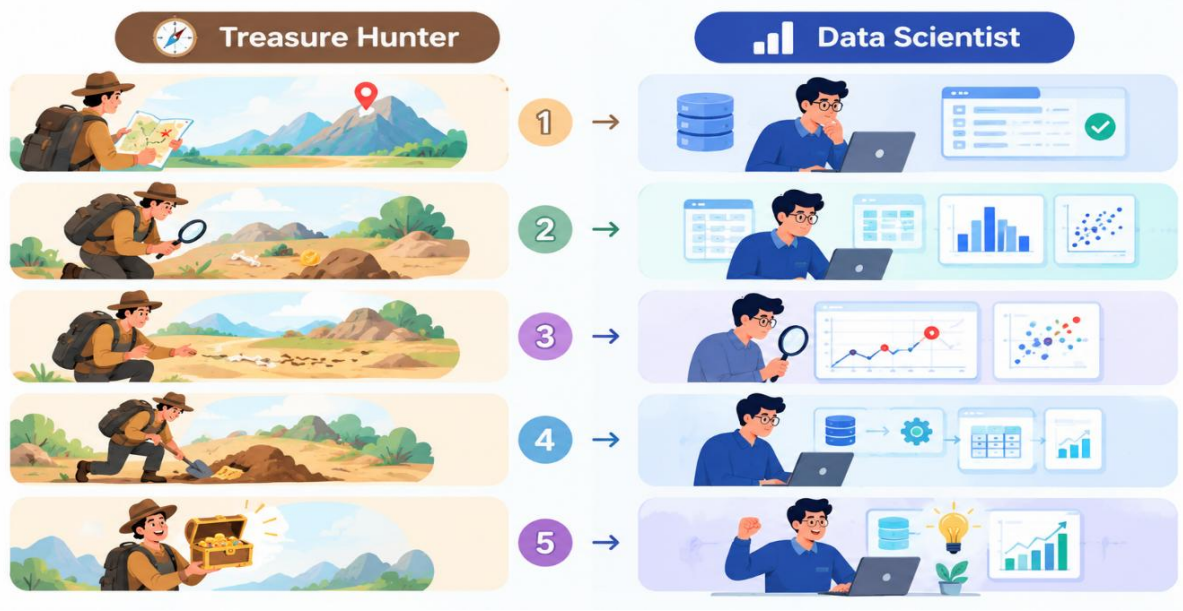
- Spot obvious errors and detect outliers
- Understand patterns within the data
- Find interesting relationships among variables
- Use the results to produce valid conclusions that can be applied to a business goal.

- Once EDA is complete, its insights can feed more advanced analysis or modelling.

EDA Analogy: Hunting for Buried Treasure

- Imagine a treasure hunter and a data scientist doing similar kinds of exploration.

Treasure Hunter	Data Scientist
Looks for a promising location	Identifies a promising dataset
Searches the area for clues	Examines the data using summaries and visualizations
Looks for patterns and unusual signs	Looks for patterns, anomalies, and relationships
Digs deeper when something looks interesting	Cleans, transforms, and explores the relevant data further
Finds the treasure	Finds useful insights



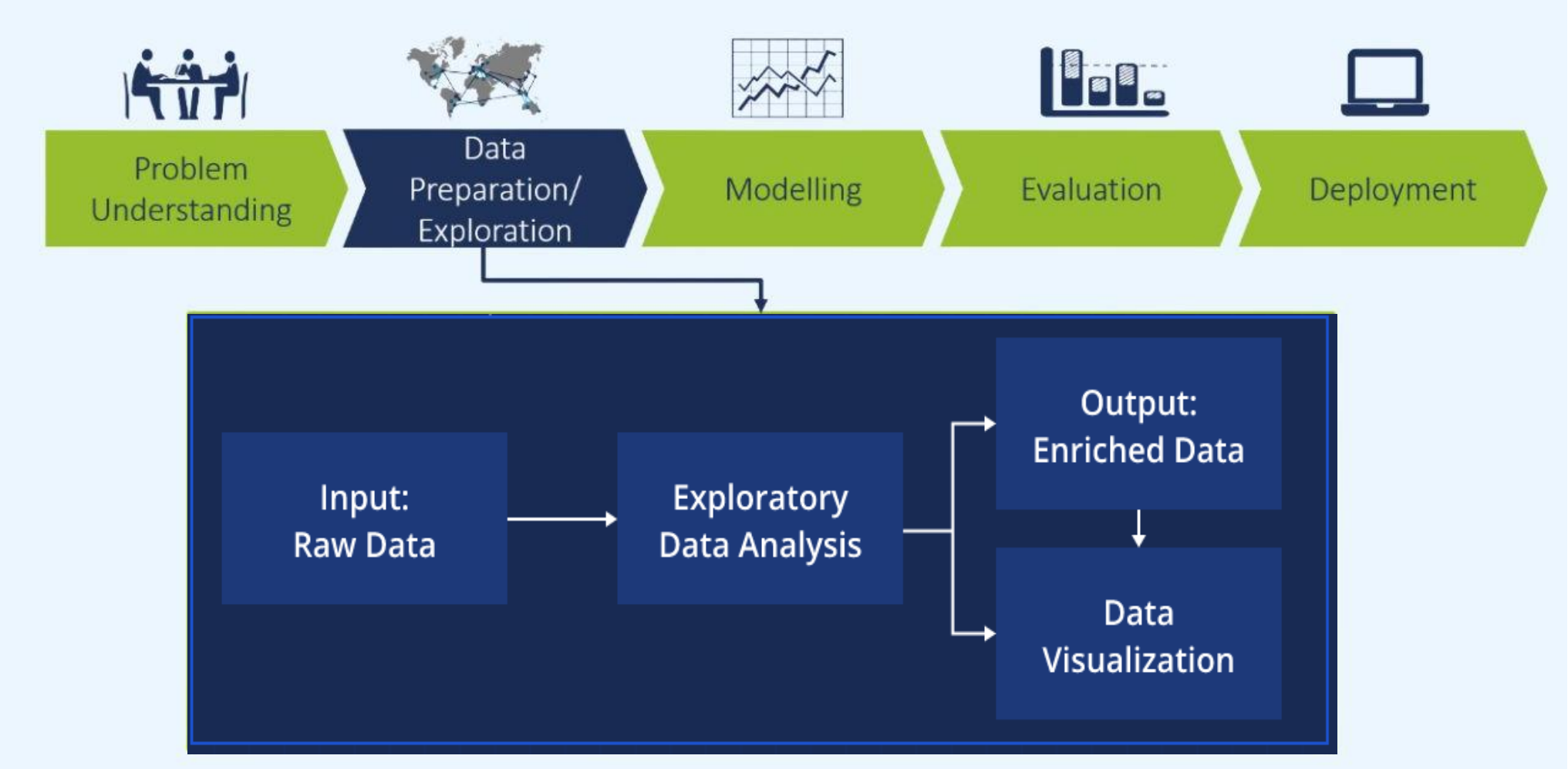
Objectives of EDA

- Understand data structure and underlying patterns.
- Identify anomalies, missing values, and outliers.
- Detect trends and relationships between variables.
- Form hypotheses to inform further analysis or modeling.

Why Do EDA?

- **Better decisions:** provides insights for data-driven decision making
- **Better models:** improves predictive model quality by identifying issues early
- **Data quality:** ensures data integrity and readiness for analysis
- **Common uses:** model building, analysis and reporting, validating assumptions, handling missing values, feature engineering, and detecting outliers

Where Does EDA Fit?



EDA is an Iterative Process

- EDA is not a one-way process. You explore, learn, and refine as you go.
- New findings often lead to new questions.
- Cleaning, transforming, or adding data can reveal new patterns.
- Keep going until you understand the data well enough for your goal.

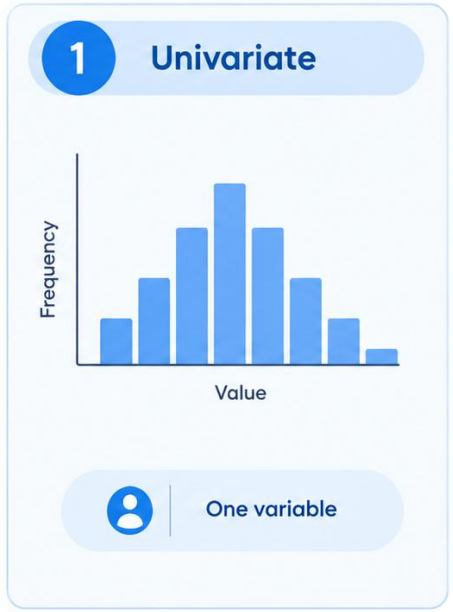


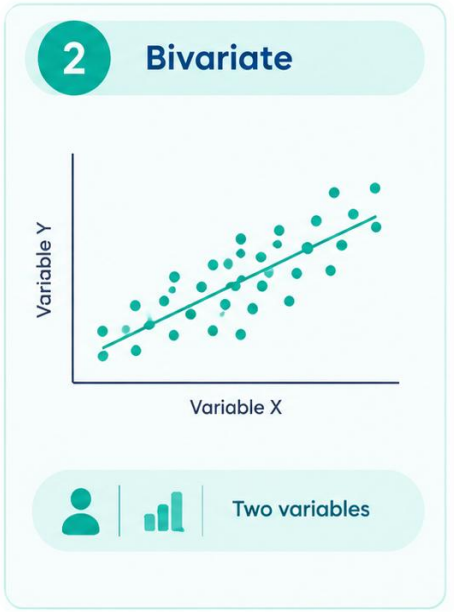
Key Steps in EDA

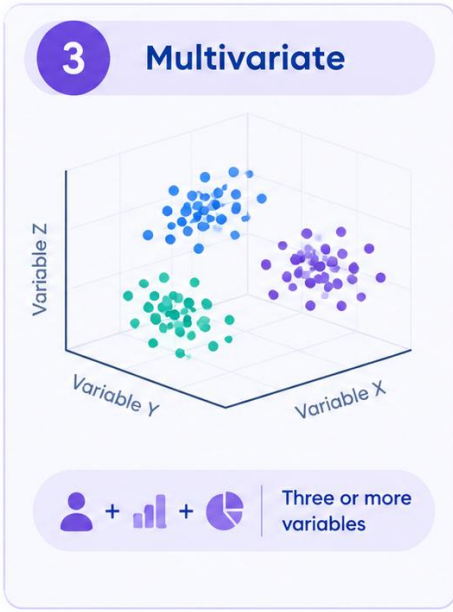
1. **Understand the problem and the data:** Rows, columns, data types. Classify each column as numerical, categorical, or mixed.
2. **Clean the data:** Handle missing values, duplicates, inconsistencies, and outliers.
3. **Univariate analysis:** Summarize each variable (mean, median, standard deviation, distribution).
4. **Bivariate analysis:** Relationships between pairs of variables.
5. **Multivariate analysis:** Interactions among several variables.
6. **Visualize:** Use charts to uncover patterns, trends and outliers (used throughout steps 3 – 5).
7. **Transform / feature engineering (if needed):** Normalize or standardize, encode categories, create new features.
8. **Document findings:** Record insights and decisions for the modelling stage.

Levels of Analysis

Level	Question	Typical tools
Univariate (one variable)	What does each variable look like?	Mean/median, histogram, box plot; counts and bar chart for categories
Bivariate (two variables)	How are two variables related?	Scatter plot, grouped box plot, cross-tabulation
Multivariate (three or more)	How do many variables interact?	Pair plot, correlation heatmap, pivot tables







Python Libraries for EDA

1. **NumPy** "Numerical Python"**:**
   - Foundation of scientific computing in Python.
   - Support large, multi-dimensional arrays and matrices.
   - Offers fast mathematical functions for numerical computation.



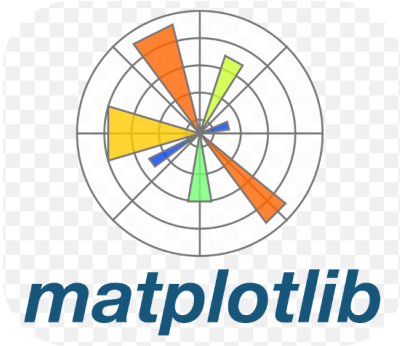
2. **Pandas:**
   - Powerful library for data manipulation and analysis
   - Provides easy-to-use data structures: `Series` and `DataFrame`
   - Ideal for cleaning, transforming, and summarizing tabular data
   - Created by Wes McKinney in 2008. The name comes from "panel data".



Python Libraries for EDA

3. Matplotlib:

- One of the oldest and most widely used Python plotting libraries (2003).
- Highly customizable, widely used for 2D plotting
- Useful for creating basic plots: `line`, `bar`, `histogram`, `scatter`, etc.



4. Seaborn:

- Built on top of Matplotlib, integrates closely with Pandas
- Simplifies the creation of complex statistical visualizations.
- Offers beautiful default styles and functions for visualizing distributions, regression, and categorical data.



5. Plotly:

- Open-source interactive graphing library for Python, R, and JavaScript.
- Enables creation of interactive, publication-quality charts and dashboards.
- Supports 3D plots, animations, and web-based visualizations.



Quick Check

1. In which phase of a data science project does EDA usually take place?
2. Name two reasons why we perform EDA before building a model.
3. In which EDA step do missing values and outliers belong: data cleaning or feature engineering?
4. Why is EDA described as an iterative process?
5. Which Python visualization library is built on top of Matplotlib?

Pandas

Python Data Analysis Library

Introduction to Pandas

Initial release: 2008 by Wes McKinney, open-sourced in 2009.

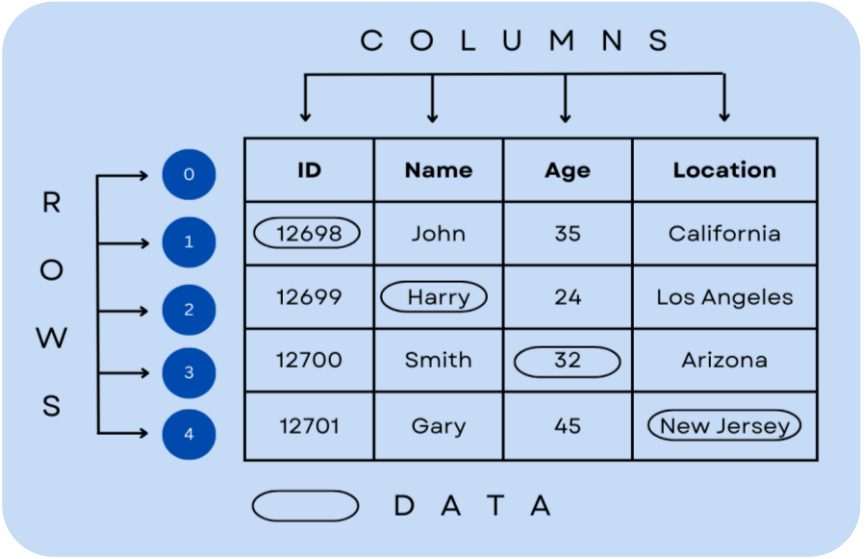
- Pandas is a powerful Python library for data manipulation and analysis.
- The name comes from **"Panel Data"** (a term for multidimensional structured datasets) and **"Python Data Analysis,"** reflecting its purpose as a library for data manipulation and analysis.
- Built on NumPy so numerical data is handled efficiently.

Why use Pandas:

- Handles structured (tabular) data efficiently.
- Easy data cleaning and transformation.
- Efficient handling of large datasets.
- Integration with other Python libraries like Matplotlib and Seaborn for visualization
- Provides the data structures like Series and DataFrame.

Getting Started: Import Pandas and Check the Version

```
import pandas as pd
print(pd.__version__)
```



Pandas Data Structures

Series:

- A 1-dimensional, array-like structure with a labeled index.
- It is used for a single column or list of data.

Code Example:

```
import pandas as pd
s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
print(s)
```

Series + Series = DataFrame

	apples
0	3
1	2
2	0
3	1

+

	oranges
0	0
1	3
2	7
3	2

=

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

```
a   10
b   20
c   30
dtype: int64
```

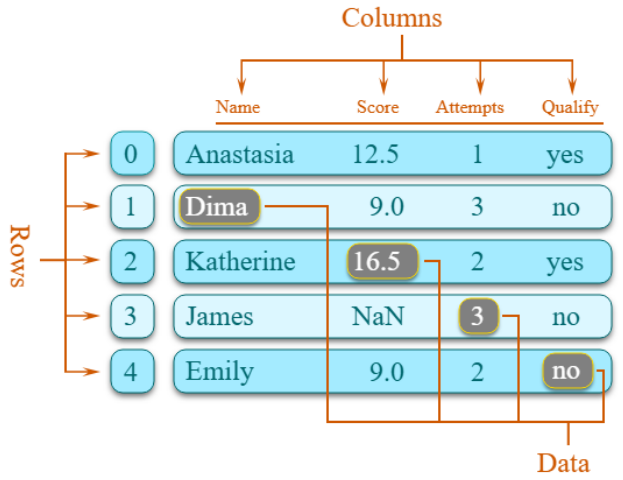
DataFrame:

- A 2-dimensional tabular structure with labeled rows and columns.
- It can be created from dictionaries, lists, or NumPy arrays.

- **Key Features**:
  - Each column can have a different data type.
  - Easy data manipulation and aggregation.
  - Methods to filter, group, and transform data efficiently.

Code Example:

```
data = {'Name': ['Alice', 'Bob'], 'Age': [25, 30]}
df = pd.DataFrame(data)
```



Pandas DataFrame

	Name	Age
0	Alice	25
1	Bob	30

Loading and Saving Data

Real EDA starts with loading a file.

```
df = pd.read_csv('data.csv')        # CSV file

df = pd.read_excel('data.xlsx')     # Excel (needs openpyxl)

df = pd.read_json('data.json')      # JSON file

df.to_csv('clean_data.csv', index=False)   # save the result
```

- Always run `df.head()` right after loading to check that the file was read correctly.

First Look at Your Data

Function	What it tells you
`df.head()`	First 5 rows / last 5 rows
`df.sample(5)`	5 random rows
`df.shape` (attribute)	(number of rows, number of columns)
`df.info()`	Column names, data types, non-null counts, memory use
`df.dtypes` (attribute)	Data type of each column
`df.describe()`	Count, mean, std, min, quartiles, max for numerical columns (include='all' for all columns)
`df['col'].value_counts()`	Frequency of each value (categorical data)
`df['col'].unique()`	The distinct values of a column
`df['col'].nunique()`	The number of distinct values

Basic Data Manipulation with Pandas

Indexing and Selection:

Pandas provides flexible tools to retrieve specific subsets of data.

- **Key Methods:**
  - `loc[]`: Access rows/columns by **labels** (row/column names).
  - `iloc[]`: Access rows/columns by **integer positions** (index numbers).

	Item	Price
0	Pen	10
1	Book	25
2	Bag	40

```
df.loc[2, 'Price']          # row with label 2, column 'Price'
df.iloc[2, 1]               # 3rd row, 2nd column
df['Price']                 # one column (a Series)
df[['Item', 'Price']]       # several columns (a DataFrame)
```

Filtering and Sorting:

```
df[df['Price'] > 20]                          # one condition
df[(df['Price'] > 20) & (df['Item'] != 'Bag')] # & (and), | (or), ~ (not)
df[df['Item'].isin(['Pen', 'Book'])]          # value in a list
df.sort_values('Price', ascending=False)      # sort high to low
```

Basic Data Manipulation with Pandas

Handling Missing Data:

Clean and prepare your data by identifying and dealing with missing values.

- **Key Functions:**
  - `isnull()` (same as `isna()`): detect missing values (return True/False). Use `.sum()` to count them per column.
  - `fillna()`: Fill missing values using a strategy (e.g., mean, median, constant)
  - `dropna()`: Remove rows (default) or columns with missing data.

	Item	Price
0	Pen	10
1	Book	25
2	Bag	40

```
df.isnull().sum()                          # missing values per column
df['Price'] = df['Price'].fillna(df['Price'].mean())   # fill with the mean
df = df.dropna()                           # drop rows with missing values
```

- Use the **mean** for roughly symmetric data. Use the **median** when the data is skewed or has outliers.
- Explore why values are missing before deciding how to handle them.

Filling in missing data

```
print(data)
```

```
     0   1   2   3
0  1.0  2.0  3.0 NaN
1  1.0  NaN  NaN NaN
2  NaN  NaN  NaN NaN
3  NaN  4.0  5.0 NaN
```

```
print(data.fillna(0))
```

```
     0   1   2   4
0  1.0  2.0  3.0  0.0
1  1.0  0.0  0.0  0.0
2  0.0  0.0  0.0  0.0
3  0.0  4.0  5.0  0.0
```

```
print(data.fillna(0, inplace=True))
print(data)
```

```
     0   1   2   4
0  1.0  2.0  3.0  0.0
1  1.0  0.0  0.0  0.0
2  0.0  0.0  0.0  0.0
3  0.0  4.0  5.0  0.0
```

Modify the dataframe instead of returning a new object (default)

```
print(data)
```

```
     0   1   2
0  1.0  2.0  3.0
1  1.0  NaN  NaN
2  NaN  NaN  NaN
3  NaN  4.0  5.0
```

```
print(data.fillna(data.mean(skipna=True)))
```

```
     0   1   2
0  1.0  2.0  3.0
1  1.0  3.0  4.0
2  1.0  3.0  4.0
3  1.0  4.0  5.0
```

replace NaN with column mean

Data Aggregation and Transformation

Grouping & Aggregating Data:

- Use `groupby()` to split data into groups based on column values and apply an aggregate operation.

Common Methods:

- `mean()`, `sum()`, `count()`, `min()`, `max()`, etc.

```python
import pandas as pd

df = pd.DataFrame({
    'Category': ['A', 'A', 'B', 'B'],
    'Sales': [100, 150, 80, 120]
})

grouped = df.groupby('Category').sum()
grouped
```

[12] ✓ 0.0s Python

Category	Sales
A	250
B	200

Applying Custom Functions:

- Use `apply()` or `transform()` to apply custom or built-in functions to rows or columns.

When to Use:

- Normalize data
- Apply conditional logic
- Add calculated features

Example – Add 5 to each value in the 'Age' column:

```python
def add_five(x):
    return x + 5

df = pd.DataFrame({'Age': [25, 30, 35]})
df['New_Age'] = df['Age'].apply(add_five)
df
```

[14] ✓ 0.0s Python

	Age	New_Age
0	25	30
1	30	35
2	35	40

Working with Dates and Times

- Pandas offers powerful tools for **date** and **time** data, which is essential for time-series analysis.

Key Features and Functions:

1. **Converting to Datetime:**
   - Use `pd.to_datetime()` to convert strings or other formats to datetime objects.
   - Support flexible parsing of various date/time formats.

```
df['Date'] = pd.to_datetime(df['Date'])
```

2. **DateTime Indexing:**
   - Set a datetime column as the index to enable time-based operations.
   - Enables easy filtering, slicing and resampling.

```
df.set_index('Date', inplace=True)
```

3. **Resmapling Time Series Data:**
   - Use `resample()` to change the frequency of observations (e.g., from daily to monthly).
   - Combine with aggregation functions like `mean()`, `sum()` or `count()`.

```
monthly_avg = df.resample('M').mean()
```

Why it Matters

- Important for trend analysis, forecasting, and seasonality detection.
- It also turns irregular time-stamped data into consistent intervals.

Advanced Data Manipulation

Merging and Joining DataFrames:

- `merge()`: SQL-style join on column(s).
- `join()`: combines DataFrames using their **index** (a shortcut for index-based merging).
- **Code Example:**

```
df1 = pd.DataFrame({'ID': [1, 2], 'Name': ['Alice', 'Bob']})
df2 = pd.DataFrame({'ID': [1, 2], 'Score': [90, 85]})
merged = pd.merge(df1, df2, on='ID')
print(merged)
```

	ID	Name	Score
0	1	Alice	90
1	2	Bob	85

Pivot Tables:

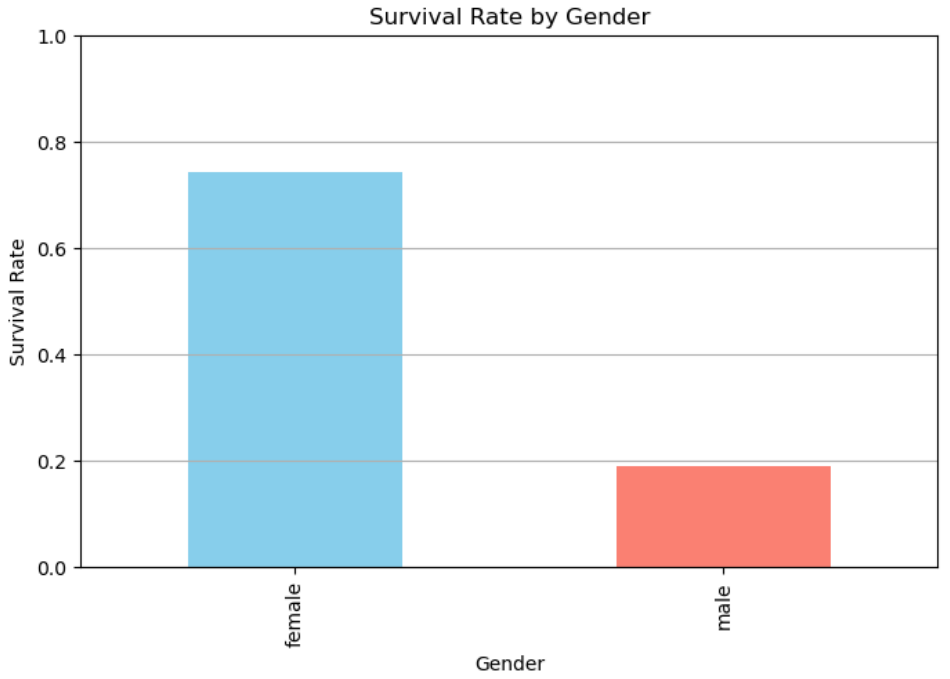
- Summarize and reshape data with `pivot_table()`.
- Aggregates data by index and columns with functions like `sum`, `mean`, etc.

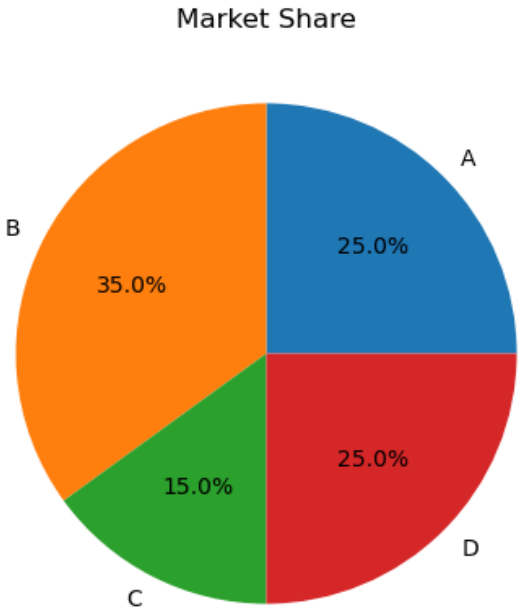
```
df = pd.DataFrame({
    'Category': ['A', 'A', 'B', 'B'],
    'Score': [85, 90, 78, 82]
})
pivot = df.pivot_table(values='Score', index='Category',
aggfunc='mean')
```

	Score
Category	
A	87.5
B	80.0

Data Visualization with Pandas

- Pandas has built-in plotting functions based on Matplotlib.
- **Features:**
  - Convenient for quick visualizations directly from DataFrames.
  - Ideal for simple, exploratory plots.
  - Compatible with Matplotlib for more customization.
- **Common Plots:**
  - **Line Plot:** Visualize trends over time.
  - **Bar Plot:** Compare categories.
  - **Histogram:** Analyze the distribution of data.
  - **Pie plot:** proportions of a whole (use sparingly)
- **Use Case**: Ideal for rapid, straightforward visual analysis during EDA.





EDA with Pandas: Putting it Together

EDA step	Pandas tools
1. Understand the data	`read_csv, head, shape, info, dtypes, describe`
2. Clean the data	`isnull().sum(), fillna, dropna, drop_duplicates, to_numeric, astype`
3. Univariate analysis	`describe, value_counts, hist, box plot`
4. Bivariate analysis	`corr(numeric_only=True), scatter plot, groupby`
5. Multivariate analysis	`pivot_table, groupby with several columns`
6. Visualize	`.plot() (plus Seaborn and Matplotlib)`
7. Transform	`apply, transform, to_datetime, merge`
8. Document and save	`to_csv`

Thank You